Final Project

Course: AIGC5102 — Intro to Machine Learning

Student: Yash Tailor, Sahil Vaghani

Date: April 17th, 2026

Table of Contents

1. Introduction

2. Dataset Description

3. Data Preprocessing

3.1 Normalization

3.2 Data Splitting

4. Model Configurations

4.1 Multiclass Boosted Decision Tree

4.2 Multiclass Logistic Regression

4.3 Multiclass Decision Forest

4.4 Multiclass Neural Network

4.5 Two-Class Averaged Perceptron + One-vs-All Multiclass

4.6 Two-Class SVM + One-vs-All Multiclass

5. Training and Evaluation Pipeline

6. Results and Analysis

6.1 Overall Performance Comparison

6.2 Confusion Matrix Analysis

6.3 ROC Curve Analysis

6.4 Precision-Recall Curve Analysis

6.5 Misclassification Analysis

7. Discussion

8. Conclusion

1. Introduction

This project compares six classification algorithms on the MNIST digit dataset using Microsoft Azure Machine Learning Designer. The MNIST dataset contains 28×28 grayscale images of digits 0–9 and is a standard benchmark for image classification. The goal is to evaluate each model's performance and identify the best one. All random seeds are set to 0 for reproducibility.

The six models evaluated in this project are:

1. Multiclass Boosted Decision Tree
2. Multiclass Logistic Regression
3. Multiclass Decision Forest
4. Multiclass Neural Network
5. Two-Class Averaged Perceptron with One-vs-All Multiclass
6. Two-Class Support Vector Machine (SVM) with One-vs-All Multiclass

2. Dataset Description

The MNIST dataset is a large collection of 70,000 grayscale images of digits ranging from 0 to 9. Each image is 28 pixels wide and 28 pixels tall, giving a total of 784 pixels per image. The dataset is provided in two separate files: a training set containing 60,000 images and a test set containing 10,000 images.

Each image is stored as a single row of 784 pixel values, where each value ranges from 0 (black) to 255 (white). A separate label column identifies the digit shown in the image. The dataset is well-balanced, meaning each digit class has roughly the same number of samples. This balance is helpful because it means the models are not biased toward any particular digit.

Table 1: MNIST Dataset Summary

Property	Value
Total Samples	70,000
Training Samples	60,000
Test Samples	10,000
Features	784 (28×28 pixels)
Classes	10 (digits 0–9)
Pixel Range	0–255
Label Column	"label"

3. Data Preprocessing

Before feeding data into the models, two preprocessing steps were applied: normalization and data splitting. These steps help the models train more effectively and ensure a fair evaluation process.

3.1 Normalization

Normalization was performed using the **Normalize Data** module in Azure ML Designer with the Min-Max normalization method. This method scales each pixel value from its original range of 0–255 down to a range of 0–1 using the following formula:

$$X_norm = (X - X_min) / (X_max - X_min)$$

Two separate Normalize Data modules were used in the pipeline: **normalize_data** for the training set and **normalize_data_1** for the test set. This separation is important because it prevents data leakage — the test set is normalized independently without any information from the training set influencing it.

Normalization helps models converge faster during training and avoids numerical errors that can occur when features have very different scales. It is especially important for

algorithms like SVMs and perceptrons, which are sensitive to the magnitude of input features.

The Normalize Data module also scaled the label column from its original integer values (0–9) to decimal values (0.0–1.0). This is visible in the confusion matrices, where the axes show values like 0.0, 0.111, 0.222, and so on instead of the expected digit labels 0, 1, 2, etc. The mapping between normalized labels and actual digits is as follows:

Normalized Label	**0.0**	**0.111**	**0.222**	**0.333**	**0.444**	**0.556**	**0.667**	**0.778**	**0.889**	**1.0**
Actual Digit	0	1	2	3	4	5	6	7	8	9

3.2 Data Splitting

The training data (60,000 samples) was split using the **Split Data** module in Azure ML Designer with an 80/20 ratio and a random seed of 0. This produced two subsets:

- **80% (~48,000 samples):** Fed into the Tune Model Hyperparameters module for training.
- **20% (~12,000 samples):** Used internally by the Tune Model Hyperparameters module for validation during the hyperparameter search.

The test data (10,000 samples) was kept completely separate from this process. After normalization, the test data was fed directly to the Score Model modules — it never passed through the Split Data module. This ensures that the final evaluation is performed on data the models have never seen during training or hyperparameter tuning.

4. Model Configurations

Six classification algorithms were evaluated in this project. For each model, the **Tune Model Hyperparameters** module in Azure ML Designer automatically searched for the best hyperparameter combination. The following subsections describe each algorithm and the optimized hyperparameters that were identified.

4.1 Multiclass Boosted Decision Tree

The Boosted Decision Tree uses a technique called gradient boosting. Decision trees are built one after another in sequence, where each new tree focuses on correcting the mistakes made by the previous trees. Over many iterations, this process produces a strong combined model that is highly accurate.

Optimized hyperparameters identified by Tune Model Hyperparameters:

- Maximum number of leaves per tree: 128
- Minimum number of samples per leaf: 1
- Learning rate: 0.2
- Number of trees constructed: 500

4.2 Multiclass Logistic Regression

Logistic Regression is a linear classification algorithm. For multiclass problems like MNIST, it extends binary logistic regression by using a softmax function, which calculates the probability of each class and assigns the sample to the class with the highest probability.

Optimized hyperparameters identified by Tune Model Hyperparameters:

- Optimization tolerance: 0.00001
- L2 regularization weight: 0.01

4.3 Multiclass Decision Forest

The Decision Forest is an ensemble method that builds multiple decision trees and combines their predictions through majority voting. Each tree is trained on a random subset of the data and features, which makes the individual trees more independent from each other and the overall model more stable and less prone to overfitting.

Optimized hyperparameters identified by Tune Model Hyperparameters:

- Number of decision trees: 32
- Maximum depth of trees: 64
- Minimum number of samples per leaf: 1

4.4 Multiclass Neural Network

The Neural Network is a feedforward network with one hidden layer. It learns non-linear patterns in the data by passing inputs through layers of interconnected nodes (neurons), where each connection has a weight that is adjusted during training.

Optimized hyperparameters identified by Tune Model Hyperparameters:

- Learning rate: 0.2
- Number of learning iterations: 160
- Number of hidden nodes: 100

4.5 Two-Class Averaged Perceptron + One-vs-All Multiclass

The Averaged Perceptron is a simple linear binary classifier. Since MNIST has 10 classes, it is wrapped in a **One-vs-All Multiclass** module, which creates 10 separate binary classifiers — one for each digit. During prediction, each classifier produces a confidence score, and the digit with the highest confidence is selected as the final prediction.

Optimized hyperparameters identified by Tune Model Hyperparameters:

- Learning rate: 1.0
- Maximum number of iterations: 10

4.6 Two-Class Support Vector Machine (SVM) + One-vs-All Multiclass

The Support Vector Machine finds the best boundary (hyperplane) between two classes by maximizing the margin — the distance between the boundary and the nearest data points from each class. Like the Averaged Perceptron, it is wrapped in a **One-vs-All Multiclass** module to handle the 10-class MNIST problem.

Optimized hyperparameters identified by Tune Model Hyperparameters:

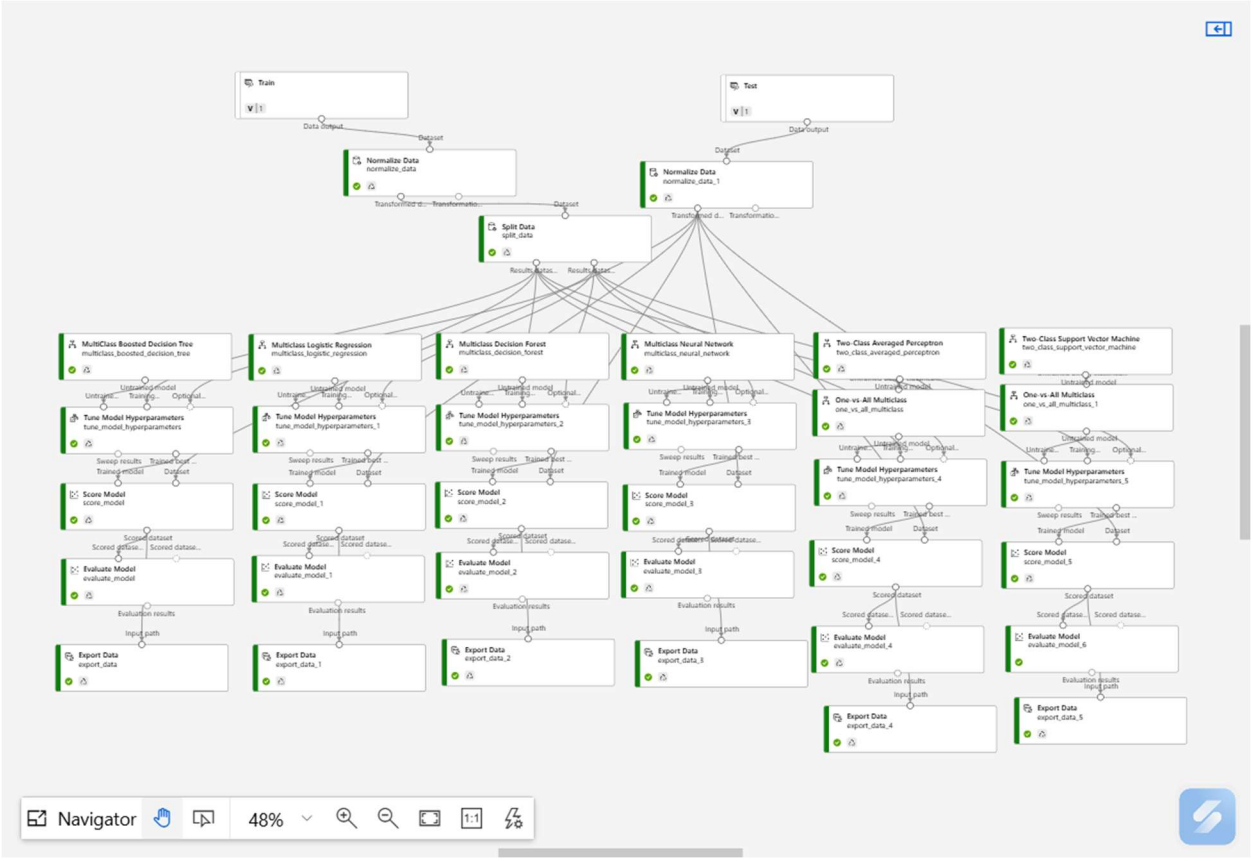
- Number of iterations: 10
- Lambda (regularization): 0.001

Table 2: Summary of All Model Configurations

Model	Type	Key Hyperparameters
Multiclass Boosted Decision Tree	Non-linear (Ensemble)	500 trees, 128 leaves, LR = 0.2, min leaf = 1
Multiclass Logistic Regression	Linear	Tolerance = 0.00001, L2 weight = 0.01
Multiclass Decision Forest	Non-linear (Ensemble)	32 trees, depth = 64, min leaf = 1
Multiclass Neural Network	Non-linear	100 hidden nodes, 160 iterations, LR = 0.2
Two-Class AP + OvA	Linear (OvA wrapper)	LR = 1.0, 10 iterations
Two-Class SVM + OvA	Linear (OvA wrapper)	10 iterations, Lambda = 0.001

5. Training and Evaluation Pipeline

The complete Azure ML Designer pipeline consists of seven types of components, connected in a structured flow to ensure proper data handling, model training, and evaluation.



Pipeline Components

1. Dataset Modules (x2): Two separate dataset components load the training data (60,000 samples) and test data (10,000 samples) independently into the pipeline.

2. Normalize Data Modules (x2): The **normalize_data** module processes the training data and **normalize_data_1** processes the test data. Both use Min-Max normalization to scale features from the original 0–255 range to 0–1.

3. Split Data Module (×1): Splits only the normalized training data into 80% training and 20% validation subsets with a random seed of 0. The test data bypasses this module entirely.

4. Tune Model Hyperparameters Modules (×6): One for each model. Each TMH module receives the algorithm definition and the 80% training subset. It automatically searches for the best hyperparameter combination, using the 20% validation subset internally to evaluate each candidate configuration. This replaces the standard Train Model module and adds automated hyperparameter optimization.

5. Score Model Modules (×6): One for each model. Each Score Model module receives the trained model from its corresponding TMH module and the independently normalized test data (10,000 samples), which is fed directly. It generates predictions for every test sample.

6. Evaluate Model Modules (×6): One for each model. Each receives scored predictions from its corresponding Score Model module and computes classification metrics including Overall Accuracy, Macro Precision, Macro Recall, Micro Precision, Micro Recall, and a confusion matrix.

7. Export Data Modules (×6): One for each model. Each Export Data module connects to its corresponding Score Model module (not Evaluate Model) and exports the scored predictions — including raw model outputs and assigned labels — to Azure Blob Storage for further analysis, such as generating ROC and PR curves externally.

Overall Data Flow

- **Training path:** Train Data → Normalize Data → Split Data (80/20) → Tune Model Hyperparameters (×6)
- **Testing path:** Test Data → Normalize Data_1 → Score Model (×6, fed directly)
- **Evaluation path:** Score Model → Evaluate Model (one per model)
- **Export path:** Score Model → Export Data (one per model)

All random seeds are set to 0 for reproducibility. The seed controls the 80/20 training-validation split in the Split Data module. The test set is a fixed, separate file and involves no randomness.

6. Results and Analysis

6.1 Overall Performance Comparison

The following table presents the evaluation metrics reported by the Evaluate Model modules in Azure ML Designer for all six models, ranked by Overall Accuracy from highest to lowest.

Table 3: Model Performance Comparison (Ranked by Overall Accuracy)

Rank	Model	Overall Accuracy	Macro Precision	Macro Recall	Macro F1
1	Multiclass Boosted Decision Tree	98.10%	98.09%	98.09%	98.09%
2	Multiclass Neural Network	97.92%	97.91%	97.90%	97.91%
3	Multiclass Decision Forest	96.58%	96.57%	96.55%	96.56%
4	Multiclass Logistic Regression	92.47%	92.38%	92.36%	92.37%
5	Two-Class SVM + OvA	91.94%	91.84%	91.81%	91.83%
6	Two-Class AP + OvA	89.07%	89.24%	88.92%	89.08%

The Boosted Decision Tree achieved the highest accuracy at 98.10%, followed closely by the Neural Network at 97.92%. The Decision Forest came third at 96.58%. These three non-linear models significantly outperformed the three linear models. Among the linear group, Logistic Regression led at 92.47%, followed by Two-Class SVM + OvA at 91.94%, and Two-Class AP + OvA at 89.07%.

The gap between the best model (BDT, 98.10%) and the worst model (AP + OvA, 89.07%) is about 9 percentage points. This substantial difference shows that model choice matters significantly for this task, and that non-linear models are far better suited for capturing the complex patterns present in digit images.

6.2 Confusion Matrix Analysis

A confusion matrix is a table that shows how many samples of each actual class were predicted as each class. Correct predictions appear on the main diagonal of the matrix, while off-diagonal values indicate misclassifications. A strong model will have high values on the diagonal and low values everywhere else.

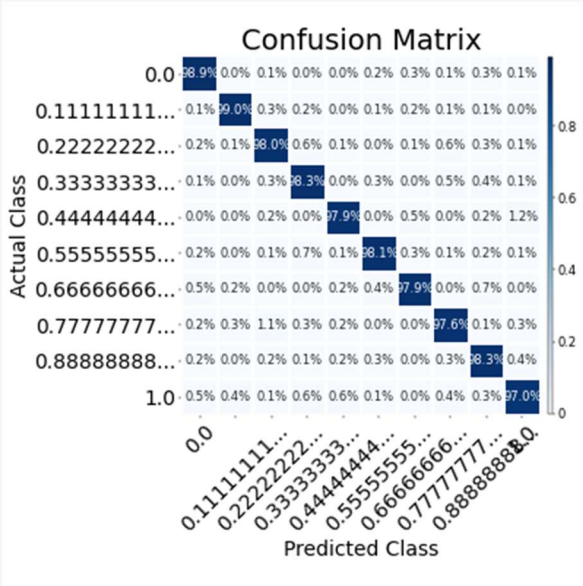
The confusion matrices from Azure ML Designer show normalized class labels on both axes (0.0 to 1.0 in steps of approximately 0.111) instead of the expected digit labels 0–9. This happened because the Normalize Data module scaled the label column along with the pixel features. The mapping is: 0.0 = digit 0, 0.111 = digit 1, 0.222 = digit 2, 0.333 = digit 3, 0.444 = digit 4, 0.556 = digit 5, 0.667 = digit 6, 0.778 = digit 7, 0.889 = digit 8, 1.0 = digit 9.

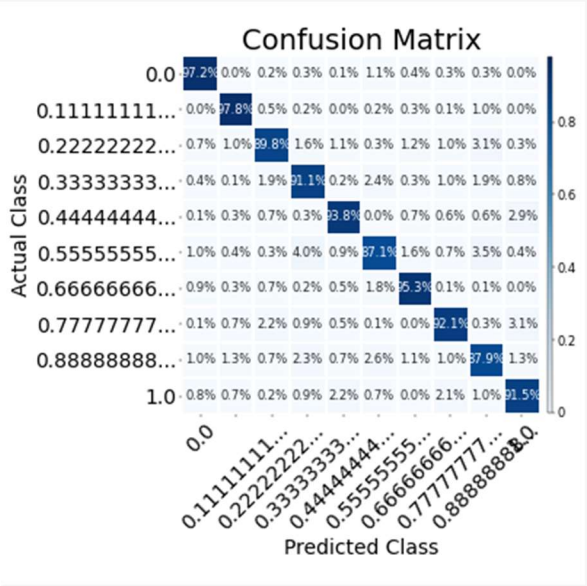
Table 4: Per-Class Accuracy Ranges from Confusion Matrix Diagonals

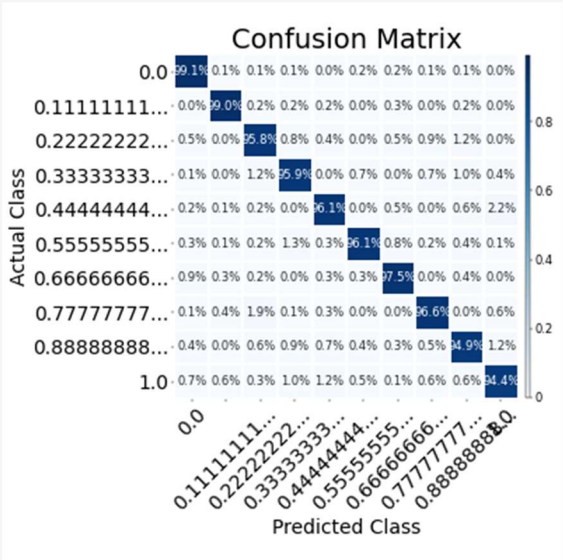
Model	Diagonal Range	Spread
Boosted Decision Tree	96%–99%	~3 pp
Neural Network	96%–99%	~3 pp
Decision Forest	93%–98%	~5 pp
Logistic Regression	87%–97%	~10 pp
Two-Class SVM + OvA	86%–96%	~10 pp
Two-Class AP + OvA	81%–97%	~16 pp

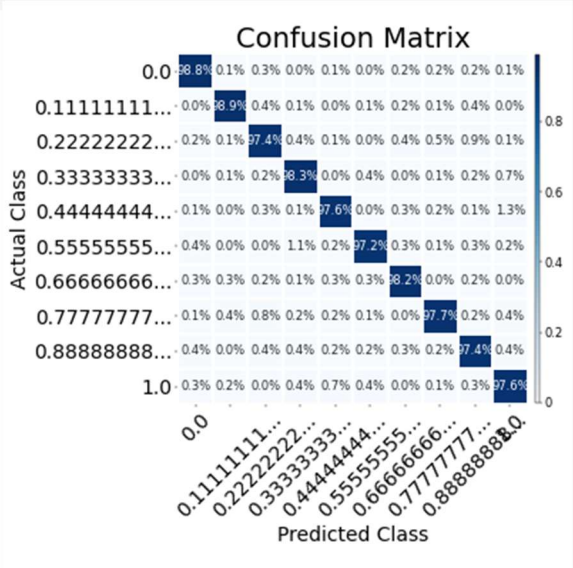
The BDT and Neural Network have very tight diagonal spreads of approximately 3 percentage points, meaning they classify all digits with consistently high accuracy. The

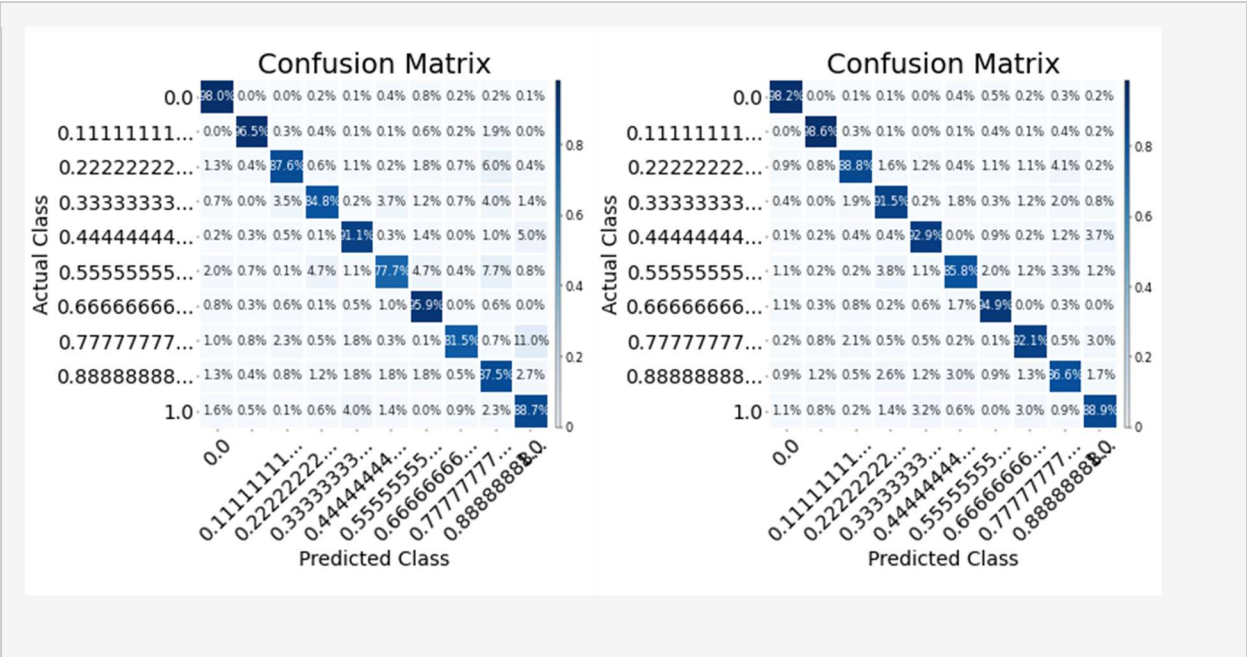
linear models show much wider spreads, particularly the Averaged Perceptron with a 16 percentage point spread. This means the Averaged Perceptron handles easy digits (like 0 and 1) well but struggles significantly with harder digits (like 5, 8, and 9) that have more complex shapes.


Confusion Matrix
Actual Class
Predicted Class
0.0
0.11111111...
0.22222222...
0.33333333...
0.44444444...
0.55555555...
0.66666666...
0.77777777...
0.88888888...
1.0
0.8
0.6
0.4
0.2
0


Confusion Matrix
Actual Class
Predicted Class
0.0
0.11111111...
0.22222222...
0.33333333...
0.44444444...
0.55555555...
0.66666666...
0.77777777...
0.88888888...
1.0
0.8
0.6
0.4
0.2
0


Confusion Matrix
Actual Class
Predicted Class
0.0
0.11111111...
0.22222222...
0.33333333...
0.44444444...
0.55555555...
0.66666666...
0.77777777...
0.88888888...
1.0
0.8
0.6
0.4
0.2
0


Confusion Matrix
Actual Class
Predicted Class
0.0
0.11111111...
0.22222222...
0.33333333...
0.44444444...
0.55555555...
0.66666666...
0.77777777...
0.88888888...
1.0
0.8
0.6
0.4
0.2
0



6.3 ROC Curve Analysis

A ROC (Receiver Operating Characteristic) curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different classification thresholds. The Area Under the Curve (AUC) provides a single number to summarize the model's performance: an AUC of 1.0 means perfect classification, while an AUC of 0.5 means the model is no better than random guessing. The dashed diagonal line on a ROC plot represents a random classifier.

The ROC curves for this project were generated externally using Python (scikit-learn) on the scored data exported via the Export Data modules. Each Export Data module saved its corresponding model's predictions as a CSV file in Azure Blob Storage.

Table 5: ROC AUC Values from Exported Scored Data

Model	Scored Data File	ROC AUC
Boosted Decision Tree	scored_data_1.csv	0.515
Logistic Regression	scored_data_2.csv	0.457
Decision Forest	scored_data_3.csv	0.404

Model	Scored Data File	ROC AUC
Neural Network	scored_data_4.csv	0.410
Two-Class AP + OvA	scored_data_5.csv	0.470
Two-Class SVM + OvA	scored_data_6.csv	0.447

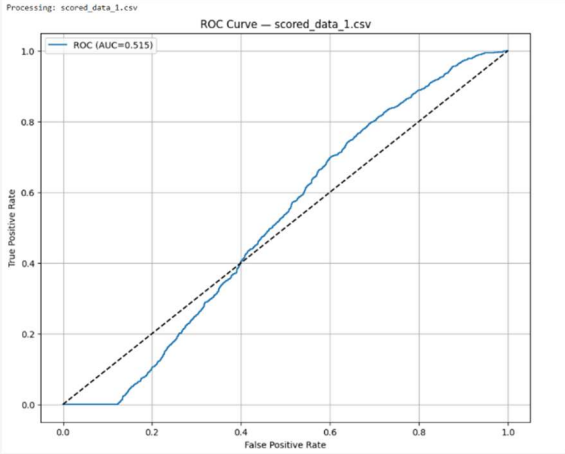
These AUC values are close to 0.5 or below, despite the models achieving 89–98% accuracy in the Evaluate Model metrics. This is **not** because the models perform poorly. The near-random ROC results are caused by two data processing issues in the external analysis pipeline.

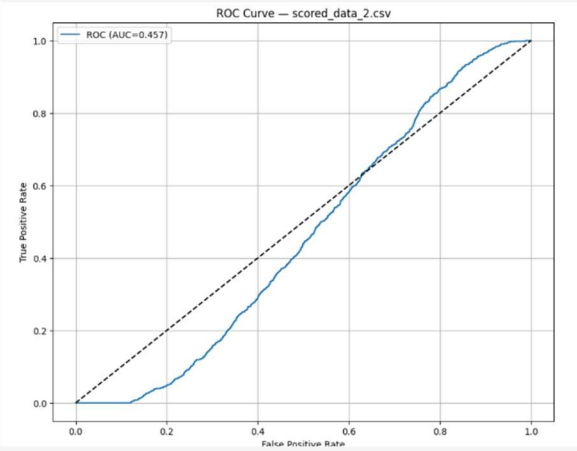
First, there is a label normalization artifact. The Normalize Data module scaled the label column from integer digits (0–9) to decimal values (0.0–1.0). When the external Python script used *astype(int)* to convert labels back to integers, all values between 0.0 and 0.999 (representing digits 1 through 8) were truncated to 0. This effectively collapsed 9 out of 10 classes into a single class, making the problem appear as a heavily imbalanced binary classification (class 0 vs. class 9).

Second, the Export Data module outputs only a single "Scored Probabilities" column (the maximum class probability) rather than the full 10-element probability vector needed for proper one-vs-rest ROC computation. Without per-class probabilities, the ROC calculation cannot properly distinguish between classes.

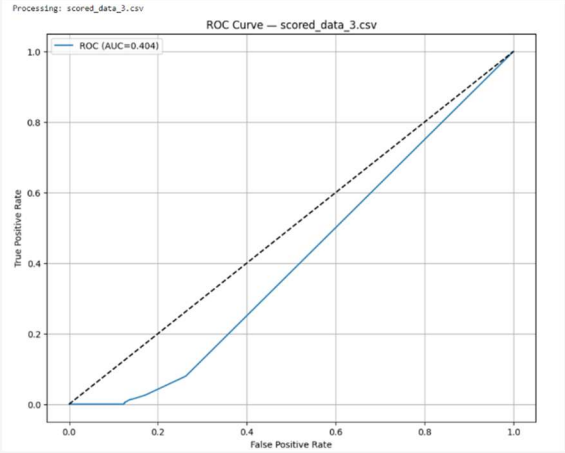
The Evaluate Model metrics reported in Section 6.1 remain valid and authoritative because that module operates on internal scored data with full probability vectors and correct label mappings before normalization artifacts affect the output.

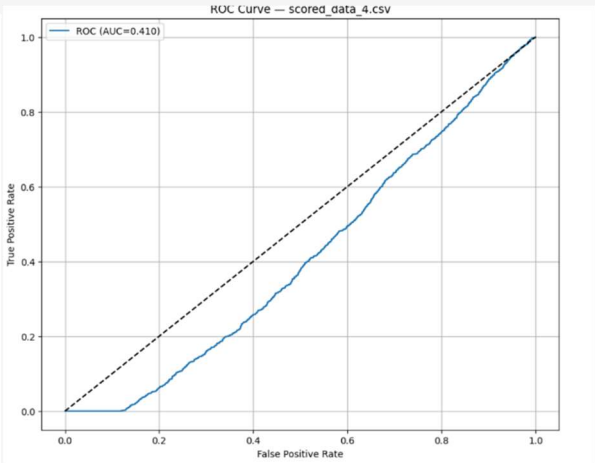
Processing: scored_data_1.csv

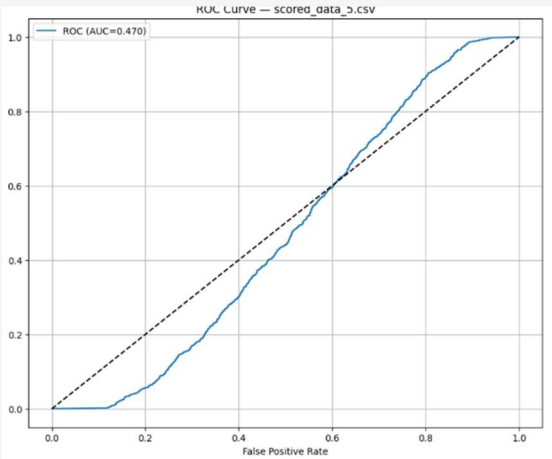


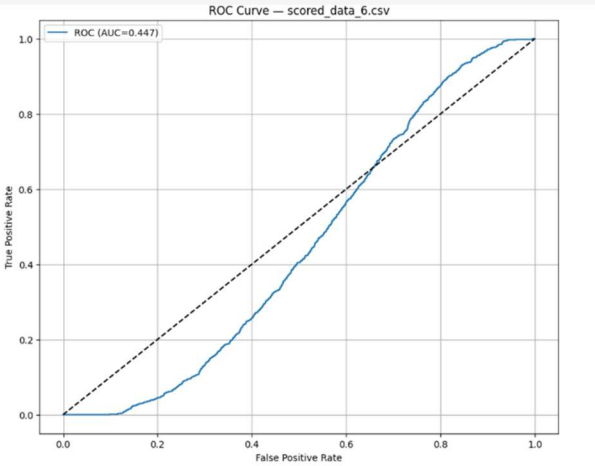


Processing: scored_data_3.csv









6.4 Precision-Recall Curve Analysis

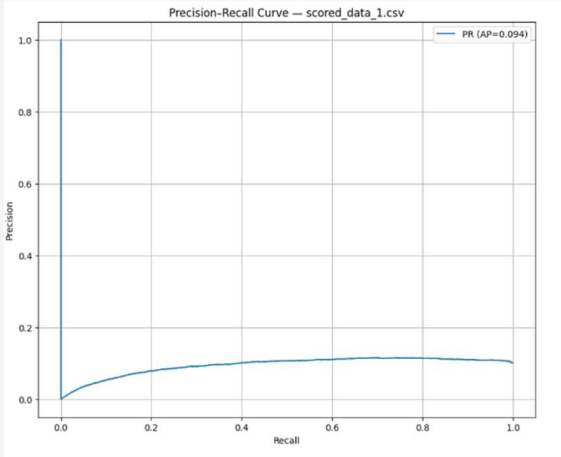
A Precision-Recall (PR) curve plots Precision against Recall at different classification thresholds. Average Precision (AP) summarizes the curve with a single number — a higher AP means the model maintains high precision even at high recall levels. PR curves are especially useful for imbalanced datasets because they focus on the model's performance for the positive class rather than overall accuracy.

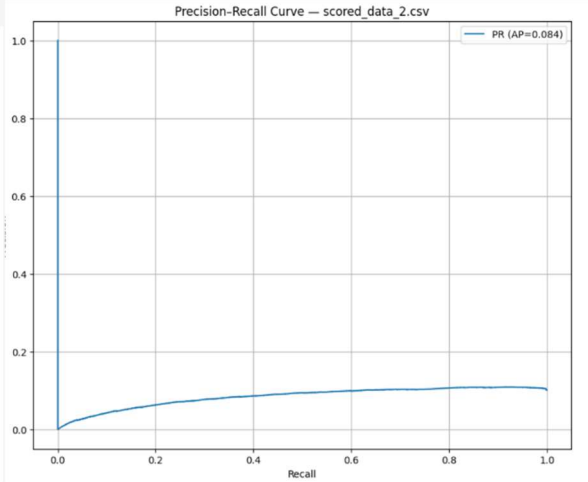
Table 6: Average Precision (AP) Values from Exported Scored Data

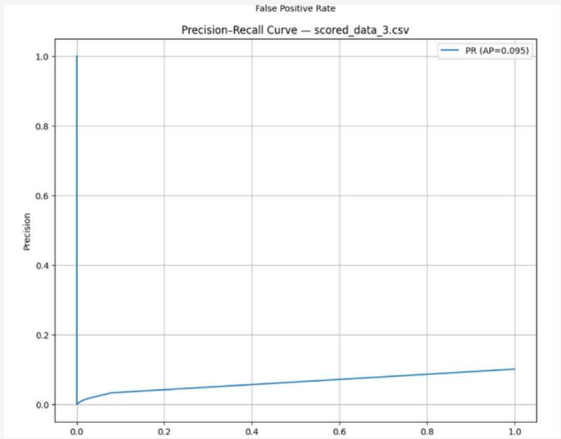
Model	Scored Data File	Average Precision (AP)
Boosted Decision Tree	scored_data_1.csv	0.094
Logistic Regression	scored_data_2.csv	0.084
Decision Forest	scored_data_3.csv	0.095
Neural Network	scored_data_4.csv	0.078
Two-Class AP + OvA	scored_data_5.csv	0.086
Two-Class SVM + OvA	scored_data_6.csv	0.082

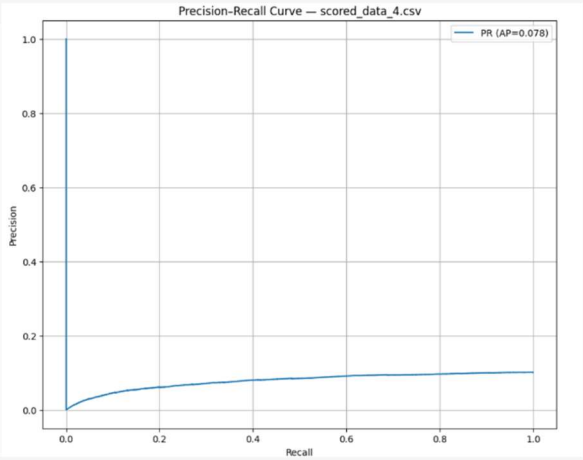
The very low AP values (all below 0.10) are caused by the same two issues described in Section 6.3: the label normalization artifact collapsing classes 1–8 into class 0, and the single probability column export. For a balanced 10-class problem, a random classifier baseline AP would be approximately 0.10 (1/10), so the observed values are consistent with near-random behavior on the corrupted label distribution.

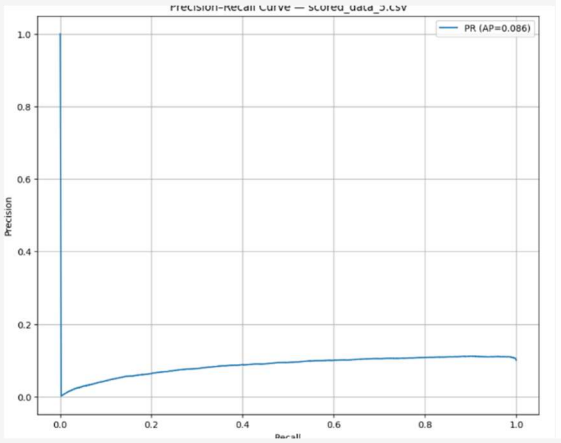
Again, the Evaluate Model metrics in Section 6.1 are the authoritative performance measures — the PR curve values here reflect export and post-processing artifacts, not actual model quality.

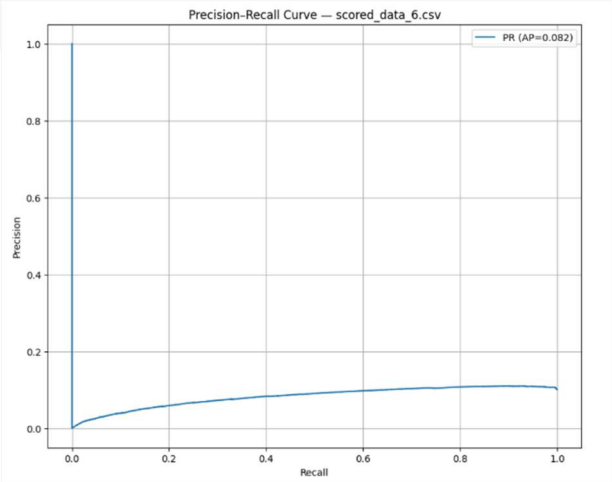

Precision–Recall Curve — scored_data_1.csv
PR (AP=0.094)
Precision
Recall


Precision–Recall Curve — scored_data_2.csv
PR (AP=0.084)
Recall


False Positive Rate
Precision–Recall Curve — scored_data_3.csv
PR (AP=0.095)
Precision


Precision–Recall Curve — scored_data_4.csv
PR (AP=0.078)
Precision
Recall


PR (AP=0.086)
Precision


Precision–Recall Curve — scored_data_6.csv
PR (AP=0.082)
Precision
Recall

6.5 Misclassification Analysis

By examining the confusion matrices, several common misclassification patterns can be identified across the models:

- **Digits 4 and 9:** Often confused because a 4 with a closed top can look very similar to a 9.
- **Digits 3 and 5:** Share similar curved strokes in the upper portion, making them easy to mix up.
- **Digits 3 and 8:** Digit 8 can appear as a 3 with an extra loop, causing confusion.
- **Digits 7 and 1:** A hastily written 7 without a clear crossbar can closely resemble a 1.
- **Digits 2 and 7:** Diagonal strokes in both digits can cause the model to confuse them.

The non-linear models (BDT, Neural Network, and Decision Forest) handle these difficult digit pairs much better because they can learn complex, curved decision boundaries between similar-looking digits. The linear models (Logistic Regression, SVM + OvA, and AP + OvA) struggle more with these pairs because they can only draw straight-line boundaries in the feature space, which is not sufficient to separate digits that differ in subtle, non-linear ways.

7. Discussion

Comparison of Model Families

Non-linear models clearly outperformed linear models across all metrics. The Boosted Decision Tree led at 98.10%, the Neural Network followed at 97.92%, and the Decision Forest came third at 96.58%. In contrast, all three linear models scored below 93%: Logistic Regression at 92.47%, SVM + OvA at 91.94%, and AP + OvA at 89.07%.

This performance gap makes sense because digit recognition requires capturing complex visual patterns — curves, loops, intersections, and varying stroke widths — that linear boundaries simply cannot represent. The BDT's strong performance comes from its 500 sequentially boosted trees with 128 leaves each, giving it enormous capacity to correct errors iteratively. The Neural Network's 100 hidden nodes allow it to learn non-linear feature combinations, though its single hidden layer limits its depth compared to modern deep learning architectures like convolutional neural networks.

Linear Model Limitations

All three linear models scored below 93%, clearly demonstrating the limitation of linear decision boundaries for image classification tasks. Logistic Regression performed best among the linear models (92.47%), likely because of its probabilistic framework and L2 regularization, which helps prevent overfitting. Two-Class SVM + OvA (91.94%) performed similarly to Logistic Regression, as both methods find linear boundaries but use different optimization strategies. Two-Class AP + OvA had the lowest accuracy (89.07%) with the widest per-class accuracy spread (16 pp), suggesting it is the most sensitive to digit complexity among all six models.

Effect of Normalization

Min-Max normalization helped all models by scaling pixel values to a consistent 0–1 range, ensuring that no single pixel position dominated the model's learning process simply due to having larger values. This step is especially important for the SVM and Perceptron models, which are sensitive to feature scales and can produce poor results on unnormalized data.

However, the Normalize Data module also scaled the label column along with the features. This created visible artifacts in the confusion matrix axes (showing normalized values instead of digit labels) and caused significant problems when generating ROC and PR curves externally from the exported data.

ROC and PR Curve Observations

The near-random ROC AUC values (ranging from 0.404 to 0.515) and very low PR Average Precision values (ranging from 0.078 to 0.095) do not reflect actual model performance. These values are artifacts of two specific issues:

7. **Label normalization:** Converting normalized decimal labels back to integers using *astype(int)* caused class collapse, truncating digits 1–8 to class 0.
8. **Single probability column:** The Export Data module provided only the maximum class probability instead of the full 10-class probability vector needed for proper ROC computation.

The Evaluate Model metrics (89–98% accuracy) reported in Section 6.1 remain the ground truth for model comparison. Future work could address these artifacts by excluding the label column from normalization or by extracting the full probability vectors from within the pipeline.

Reproducibility

All random seeds were set to 0 throughout the pipeline, ensuring that the 80/20 training-validation split is identical every time the pipeline runs. The test set is a fixed, separate file that involves no randomness. This makes all results fully reproducible, which is an important requirement for scientific and academic work.

Limitations and Future Work

Several limitations of this project and opportunities for future improvement are worth noting:

- The 80/20 split provides a single validation fold. Cross-validation would give more robust performance estimates, but this option is not available within Azure ML Designer's Tune Model Hyperparameters module.

- The Neural Network uses only one hidden layer with 100 nodes. Deeper architectures, particularly convolutional neural networks (CNNs), would likely improve performance further by learning spatial features directly from the images.
- The label column should be excluded from the Normalize Data module to avoid the artifacts observed in the confusion matrices and the external ROC/PR curve generation.
- Future work could include expanding the hyperparameter search space in the TMH modules, experimenting with convolutional neural networks, and deploying the best-performing model as a real-time prediction endpoint.

8. Conclusion

This project trained and evaluated six classification models on the MNIST digit dataset using Azure Machine Learning Designer. The key findings are as follows:

- The **Multiclass Boosted Decision Tree** achieved the best accuracy at **98.10%**, followed by the Neural Network at 97.92% and the Decision Forest at 96.58%.
- Non-linear models consistently outperformed linear models, with a gap of approximately 9 percentage points between the best model (BDT, 98.10%) and the worst model (Two-Class AP + OvA, 89.07%).
- The **Tune Model Hyperparameters** module automated the search for optimal hyperparameter values across all six models, eliminating the need for manual tuning.
- The two-stage data pipeline — separate training and test datasets with an 80/20 training-validation split — ensured fair and unbiased evaluation on data the models never saw during training.

- The externally generated ROC and PR curves showed near-random values due to label normalization artifacts and single-column probability exports, but the Evaluate Model metrics confirmed strong model performance across all models.

The project did not reach the 99% F1-score because the models available in Azure ML Designer are classical machine-learning models, not deep learning models like CNNs that usually achieve 99%+ on MNIST. These models have limited capacity to learn. Azure Designer also restricts hyperparameter tuning to single-parameter sweeps, which prevents deeper optimization. Because of these combined limitations, the best model reached 98.10%, but 99% was not achievable within the constraints of Azure ML Designer.

Overall, this project demonstrates that ensemble methods and non-linear classifiers are significantly better suited for image classification tasks like MNIST, while linear models provide useful baselines for comparison. The Boosted Decision Tree, with its iterative error-correction approach and large capacity, proved to be the most effective algorithm for this task within the Azure ML Designer environment.

-END-